UNIVERSIDAD TECNOLÓGICA

TULA-TEPEJI

Tecnologías de la Información

Desarrollo De Software Multiplataforma

Estructura de datos

Ejercicios

Desarrollado por:

- Adair Jesús Cruz Huitrón 25300767 G1

Edith Campos Cruz

A 4 de Octubre, 2026

1. Análisis de calificaciones

Desarrolla un programa que almacene las calificaciones de **30 estudiantes** en un arreglo.

El programa deberá:

- Capturar las 30 calificaciones.
- Validar que estén entre 0 y 10.
- Calcular el promedio del grupo.
- Determinar la calificación mayor y menor.
- Mostrar cuántos estudiantes:
    - Aprobaron.
    - Reprobaron.
    - Obtuvieron una calificación superior al promedio.
- Mostrar las posiciones de los estudiantes que obtuvieron la calificación máxima.
- Crear un segundo arreglo que contenga únicamente las calificaciones aprobatorias.

- Ordenar el segundo arreglo de menor a mayor.

Pseudocódigo:

INICIO

DEFINIR ALUMNOS = 30

DEFINIR arreglo cal[ALUMNOS]

DEFINIR arreglo salvados

PARA i = 0 HASTA ALUMNOS - 1 HACER

REPETIR

MOSTRAR "Ingresa la calificación del alumno"

LEER calificacion

INTENTAR convertir calificacion a número

SI la conversión es correcta ENTONCES

SI calificacion >= 0 Y calificacion <= 10 ENTONCES

valor = calificacion

SI NO

valor = -1

FIN SI

SI NO

MOSTRAR "Solo números entre 0 y 10"

valor = -1

FIN SI

HASTA QUE valor != -1

cal[i] = valor

FIN PARA

CONTAR cantidad de alumnos con calificación >= 7

CREAR arreglo salvados con tamaño igual a cantidad de aprobados

DEFINIR x = 0

PARA i = 0 HASTA ALUMNOS - 1 HACER

SI cal[i] >= 7 ENTONCES

salvados[x] = cal[i]

x = x + 1

FIN SI

FIN PARA

PARA i = 1 HASTA tamaño de salvados - 1 HACER

key = salvados[i]

j = i - 1

MIENTRAS j >= 0 Y salvados[j] > key HACER

salvados[j + 1] = salvados[j]

j = j - 1

FIN MIENTRAS

salvados[j + 1] = key

FIN PARA

CALCULAR promedio

CALCULAR calificación mayor

CALCULAR calificación menor

CONTAR alumnos aprobados

CONTAR alumnos reprobados

CONTAR alumnos con calificación superior al promedio

BUSCAR posiciones de alumnos con calificación 10

MOSTRAR calificaciones aprobatorias ordenadas

FIN

Código:

alumnos.java:

```
package Unidad_1.Tarea.Ej_1;

import javax.swing.JOptionPane;

public class alumnos {
int ALUMNOS = 30;
double cal[];
double salvados[];
public alumnos(int aLUMNOS) {
ALUMNOS = aLUMNOS;
this.cal = new double[ALUMNOS];
}
public void llenar_arreglo(){
for (int i = 0; i < ALUMNOS; i++) {
double v = 0;
do {
```

```
String n = JOptionPane.showInputDialog("Ingresa la calificación del alumno: " + (i + 1));
v = validar(n);
} while (v == -1);
cal[i] = v;
}
}

public double validar(String n) {
try {
double x = Double.parseDouble(n);
if (x <= 10.0 && x >= 0.0) {
return x;
} else {
return -1;
}
} catch (Exception e) {
JOptionPane.showMessageDialog(null,"Solo números entre 0 y 10");
return -1;
}
}
public double mayor(){
double m=0;
for(int i=0;i<ALUMNOS;i++){
if(m<cal[i]){
m=cal[i];
}
}
return m;
}
public double menor(){
double m=cal[0];
for(int i=0;i<ALUMNOS;i++){
if(m>cal[i]){
m=cal[i];
}
}
return m;
}
public int aprobaron(){
int m=0;
for(int i=0;i<ALUMNOS;i++){
if(cal[i]>=7.0){
m++;
}
}
```

```
return m;
}

public int reprobaron(){
int m=0;
for(int i=0;i<ALUMNOS;i++){
if(cal[i]<7.0){
m++;
}
}
return m;
}
public double promedio(){
double m=0;
for(int i=0;i<ALUMNOS;i++){
m+=cal[i];
}
return (m/ALUMNOS);
}
public int mayPromedio(){
int m=0;
double prom = promedio();
for(int i=0;i<ALUMNOS;i++){
if(cal[i]>prom){
m++;
}
}
return m;
}

public String maximos(){
String m="";
double prom = promedio();
for(int i=0;i<ALUMNOS;i++){
if(cal[i]==10){
m+= " ["+ i +"] \n";
}
}
return m;
}
public void aprobados(){
this.salvados = new double[aprobaron()];
for (int i = 0, x = 0; i < ALUMNOS; i++) {
if (cal[i] >= 7.0) {
salvados[x] = cal[i];
```

```
x++;
}
}
}

public void ordenar(){
int n = salvados.length;
for (int i = 1; i < n; ++i) {
double key = salvados[i];
int j = i - 1;
while (j >= 0 && salvados[j] > key) {
salvados[j + 1] = salvados[j];
j = j - 1;
}
salvados[j + 1] = key;
}
}
public String mostrarAprobados() {

String m = "";

for (int i = 0; i < salvados.length; i++) {
m += salvados[i] + " ";
}

return m;
}
}
```

main.java

```
package Unidad_1.Tarea.Ej_1;


public class main {

public static void main(String[] args) {

alumnos grupo = new alumnos(30);

grupo.llenar_arreglo();

grupo.aprobados();
```

```
grupo.ordenar();

System.out.println("===== RESULTADOS DEL GRUPO =====");
System.out.println("Promedio del grupo: " + String.format("%.2f", grupo.promedio()));
System.out.println("Calificación mayor: " + grupo.mayor());
System.out.println("Calificación menor: " + grupo.menor());
System.out.println("Estudiantes que aprobaron: " + grupo.aprobaron());
System.out.println("Estudiantes que reprobaron: " + grupo.reprobaron());
System.out.println("Estudiantes con calificación superior al promedio: " +
grupo.mayPromedio());
System.out.println("Posiciones con calificación máxima:");
System.out.println(grupo.maximos());
System.out.println("Calificaciones aprobatorias ordenadas:");
System.out.println(grupo.mostrarAprobados());
}
}
```

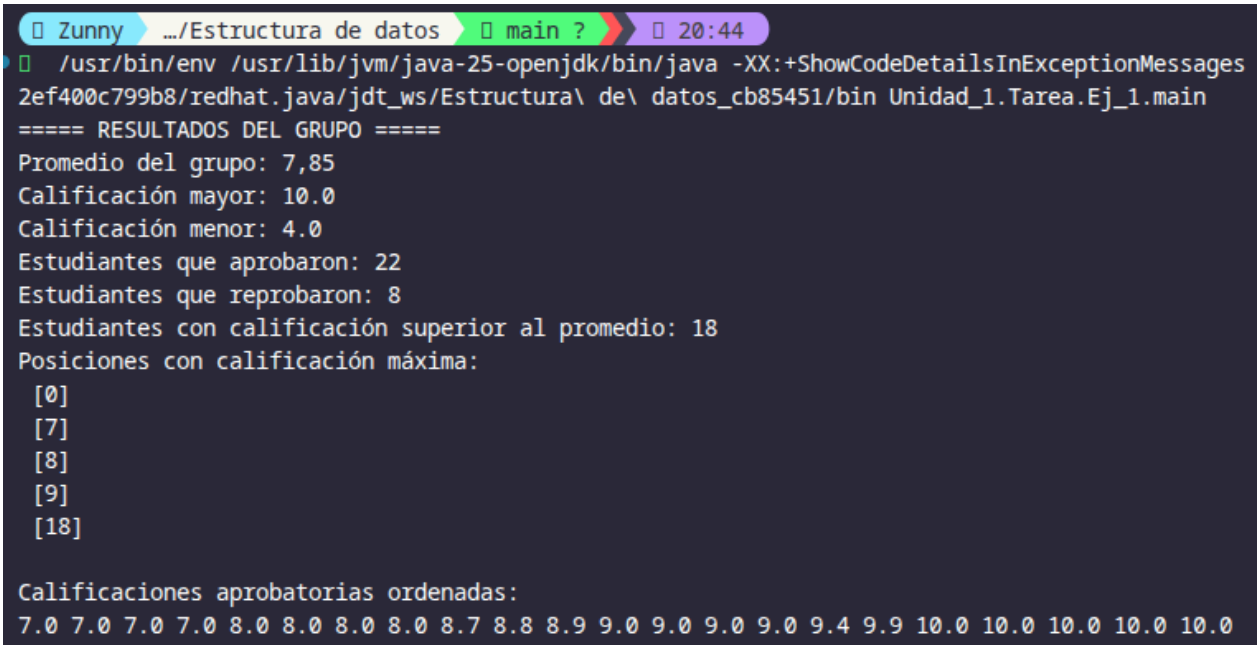
Datos de prueba

- 10
- 9
- 8
- 7
- 6
- 5
- 4
- 10
- 10
- 10

- 9
- 9
- 9
- 8
- 8
- 8
- 9.9
- 9.4
- 10
- 6.9

- 6.9
- 7
- 7
- 7
- 5
- 5
- 5
- 8.7
- 8.8
- 8.9

Captura:



2. Eliminación de elementos duplicados

Dado un arreglo de **20 números enteros**, desarrolla un programa que:

- Permita capturar los valores.
- Identifique los números repetidos.
- Determine cuántas veces aparece cada número.
- Genere un segundo arreglo que contenga únicamente los valores **sin duplicados**.
- Muestre cuál fue el número que más veces apareció.

- Ordene el arreglo resultante de menor a mayor.

Pseudocódigo:

INICIO

DEFINIR NUMEROS = 20

CREAR arreglo num[NUMEROS]

CREAR arreglo unicos[NUMEROS]

DEFINIR cantidadUnicos

```
// Llenar arreglo
PARA i = 0 HASTA NUMEROS - 1 HACER
    REPETIR
        MOSTRAR "Ingresa el número de la posición"
        LEER número
        INTENTAR convertir número a entero
        SI la conversión es correcta ENTONCES
            valor = número
        SI NO
            MOSTRAR "Solo números"
            valor = -1
        FIN SI
    HASTA QUE valor != -1
    num[i] = valor
FIN PARA
// Buscar números repetidos
rep = ""
PARA i = 0 HASTA NUMEROS - 1 HACER
    PARA o = i + 1 HASTA NUMEROS - 1 HACER
        SI num[i] == num[o] ENTONCES
            AGREGAR num[i] a rep
        FIN SI
    FIN PARA
FIN PARA
// Contar cuántas veces aparece cada número
rep = ""
```

```
PARA i = 0 HASTA NUMEROS - 1 HACER
    repe = 0
    PARA o = 0 HASTA NUMEROS - 1 HACER
        SI num[i] == num[o] ENTONCES
            repe = repe + 1
        FIN SI
    FIN PARA
    AGREGAR número y cantidad de repeticiones a rep
FIN PARA
// Eliminar duplicados
CREAR arreglo unicos[NUMEROS]
x = 0
PARA i = 0 HASTA NUMEROS - 1 HACER
    yaEsta = FALSO
    PARA o = 0 HASTA i - 1 HACER
        SI num[i] == num[o] ENTONCES
            yaEsta = VERDADERO
            SALIR DEL CICLO
        FIN SI
    FIN PARA
    SI yaEsta == FALSO ENTONCES
        unicos[x] = num[i]
        x = x + 1
    FIN SI
FIN PARA
cantidadUnicos = x
```

```
// Encontrar número más repetido
vecesmax = 0
nummax = 0
PARA i = 0 HASTA NUMEROS - 1 HACER
    numero = num[i]
    veces = 0
    PARA o = i HASTA NUMEROS - 1 HACER
        SI num[i] == num[o] ENTONCES
            veces = veces + 1
        FIN SI
    FIN PARA
    SI veces > vecesmax ENTONCES
        vecesmax = veces
        nummax = numero
    FIN SI
FIN PARA
// Ordenar arreglo sin duplicados
PARA i = 1 HASTA tamaño de unicos - 1 HACER
    key = unicos[i]
    j = i - 1
    MIENTRAS j >= 0 Y unicos[j] > key HACER
        unicos[j + 1] = unicos[j]
        j = j - 1
    FIN MIENTRAS
    unicos[j + 1] = key
FIN PARA
```

// Mostrar resultados

MOSTRAR arreglo original

MOSTRAR números repetidos

MOSTRAR cuántas veces aparece cada número

MOSTRAR número más repetido

MOSTRAR arreglo sin duplicados y ordenado

FIN

Codigps:

Arreglo.java

```
package Unidad_1.Tarea.Ej_2;


import javax.swing.JOptionPane;

public class arreglo {
int NUMEROS = 20;
int num[];
int unicos[];
int cantidadUnicos;
public arreglo(int nUMEROS) {
NUMEROS = nUMEROS;
this.num = new int[NUMEROS];
}
public void llenar_arreglo(){
for (int i = 0; i < NUMEROS; i++) {
int v = 0;
do {
String n = JOptionPane.showInputDialog("Ingresa le numero de la posición: " + (i + 1));
v = validar(n);
} while (v == -1);
num[i] = v;
}
}

public int validar(String n) {
try {
int x = Integer.parseInt(n);
return x;
```

```
} catch (Exception e) {
JOptionPane.showMessageDialog(null,"Solo números");
return -1;
}
}

// no puedo pensar en uno que no sea n²
public String repetidos(){
String rep="";
for(int i=0;i< NUMEROS;i++){
for(int o=i+1; o<NUMEROS;o++){
if(num[i]==num[o]){
rep+= num[i]+" ";
}
}
}
return rep;
}

public String contarRepetidos(){
String rep="";
for(int i=0;i< NUMEROS;i++){
int repe=0;
for(int o=0; o<NUMEROS;o++){
if(num[i]==num[o]){
repe++;
}
}
rep+="Numero: " + num[i] + " : " + repe + " vececs\n";
}
return rep;
}
public void sinDuplicados(){
this.unicos = new int[NUMEROS];
int x=0;
for(int i=0;i<NUMEROS;i++){
boolean yaEsta=false;

for(int o=0;o<i;o++){
if(num[i]==num[o]){
yaEsta=true;
break;
}
}
```

```
if(yaEsta){
continue;
}else{
unicos[x] = num[i];
x++;
}

}
cantidadUnicos = x;
}
public int numeroFamoso(){
int numero;
int veces;
int vecesmax = 0;
int nummax = 0;
for(int i=0;i< NUMEROS;i++){
numero = num[i];
veces=0;
for(int o=i; o<NUMEROS;o++){
if(num[i]==num[o]){
veces++;
}
}
if(veces>vecesmax){
vecesmax = veces;
nummax = numero;
}
}
return nummax;
}

public void ordenar(){
int n = unicos.length;
for (int i = 1; i < n; ++i) {
int key = unicos[i];
int j = i - 1;
while (j >= 0 && unicos[j] > key) {
unicos[j + 1] = unicos[j];
j = j - 1;
}
unicos[j + 1] = key;
}
}
public String mostrarOriginal() {
```

```
String m = "";

for (int i = 0; i < num.length; i++) {
m += num[i] + " ";
}

return m;
}
public String mostrarUnicos() {

String m = "";

for (int i = 0; i < cantidadUnicos; i++) {
m += unicos[i] + " ";
}

return m;
}
}
```

main.java

```
package Unidad_1.Tarea.Ej_2;


public class main {

public static void main(String[] args) {

arreglo numeros = new arreglo(20);

numeros.llenar_arreglo();

numeros.sinDuplicados();

numeros.ordenar();

System.out.println("===== ARREGLO ORIGINAL =====");
System.out.println(numeros.mostrarOriginal());

System.out.println("===== NÚMEROS REPETIDOS =====");
System.out.println(numeros.repetidos());

System.out.println("===== VECES QUE APARECE CADA NÚMERO =====");
```

```
System.out.println(numeros.contarRepetidos());

System.out.println("===== NÚMERO MÁS REPETIDO =====");
System.out.println("Número: " + numeros.numeroFamoso());

System.out.println("===== ARREGLO SIN DUPLICADOS ORDENADO =====");
System.out.println(numeros.mostrarUnicos());
}
}
```

Datos de prueba

- 77
- 55
- 33
- 2
- 7
- 8
- 99
- 99
- 9
- 34
- 67
- 69
- 81
- 24
- 61
- 1
- 5
- 82
- 36
- 7

Resultados

```
 Zunny …/Estructura de datos  main ?  20:48
 cd /home/Zunny/Documentos/UTTT/4°\ CUATRI/Estructura\ 
home/Zunny/.config/Code/User/workspaceStorage/0a465d4da33
===== ARREGLO ORIGINAL =====
77 55 33 2 7 8 99 9 9 34 67 69 81 24 61 1 5 82 36 7
===== NÚMEROS REPETIDOS =====
7 9
===== VECES QUE APARECE CADA NÚMERO =====
Numero: 77 : 1 vececs
Numero: 55 : 1 vececs
Numero: 33 : 1 vececs
Numero: 2 : 1 vececs
Numero: 7 : 2 vececs
Numero: 8 : 1 vececs
Numero: 99 : 1 vececs
Numero: 9 : 2 vececs
Numero: 9 : 2 vececs
Numero: 34 : 1 vececs
Numero: 67 : 1 vececs
Numero: 69 : 1 vececs
Numero: 81 : 1 vececs
Numero: 24 : 1 vececs
Numero: 61 : 1 vececs
Numero: 1 : 1 vececs
Numero: 5 : 1 vececs
Numero: 82 : 1 vececs
Numero: 36 : 1 vececs
Numero: 7 : 2 vececs

===== NÚMERO MÁS REPETIDO =====
Número: 7
===== ARREGLO SIN DUPLICADOS ORDENADO =====
0 0 1 2 5 7 8 9 24 33 34 36 55 61 67 69 77 81
```

3. Análisis de calificaciones por grupo

Una universidad tiene **4 grupos con 6 estudiantes cada uno**.

Utiliza una matriz:

calificaciones[4][6]

Cada fila representa un grupo y cada columna un estudiante.

El programa deberá:

- Capturar las calificaciones.
- Calcular el promedio de cada grupo.
- Calcular el promedio general.
- Determinar la calificación más alta de toda la matriz.
- Indicar grupo y estudiante que obtuvieron dicha calificación.
- Contar cuántos estudiantes aprobaron y reprobaron.
- Determinar cuál grupo obtuvo el promedio más alto.

- Mostrar a los estudiantes que están por encima del promedio general.

Pseudocódigo:

```
INICIO
    DEFINIR GRUPOS = 4
    DEFINIR ESTUDIANTES = 6
    CREAR matriz esc[GRUPOS][ESTUDIANTES]
    // Llenar matriz
    PARA i = 0 HASTA GRUPOS - 1 HACER
        PARA o = 0 HASTA ESTUDIANTES - 1 HACER
            REPETIR
                MOSTRAR "Ingresa la calificación del alumno"
                LEER calificación
```

```
            INTENTAR convertir la calificación a número
            SI la calificación está entre 0 y 10 ENTONCES
               valor = calificación
            SI NO
               MOSTRAR "Solo números entre 0 y 10"
               valor = -1
            FIN SI
         HASTA QUE valor != -1
         esc[i][o] = valor
      FIN PARA
   FIN PARA
   // Promedio de cada grupo
   PARA i = 0 HASTA GRUPOS - 1 HACER
      suma = 0
      PARA o = 0 HASTA ESTUDIANTES - 1 HACER
         suma = suma + esc[i][o]
      FIN PARA
      promedio = suma / ESTUDIANTES
      MOSTRAR promedio del grupo
   FIN PARA
   // Promedio general
   suma = 0
   PARA i = 0 HASTA GRUPOS - 1 HACER
      suma = suma + promedio del grupo i
   FIN PARA
   promedioGeneral = suma / GRUPOS
```

```
MOSTRAR promedioGeneral
// Calificación máxima
max = 0
grupo = 0
estudiante = 0
PARA i = 0 HASTA GRUPOS - 1 HACER
    PARA o = 0 HASTA ESTUDIANTES - 1 HACER
        SI max < esc[i][o] ENTONCES
            max = esc[i][o]
            grupo = i
            estudiante = o
        FIN SI
    FIN PARA
FIN PARA
MOSTRAR max
MOSTRAR grupo
MOSTRAR estudiante
// Contar aprobados
aprobados = 0
PARA i = 0 HASTA GRUPOS - 1 HACER
    PARA o = 0 HASTA ESTUDIANTES - 1 HACER
        SI esc[i][o] >= 7 ENTONCES
            aprobados = aprobados + 1
        FIN SI
    FIN PARA
FIN PARA
```

```
MOSTRAR aprobados
// Contar reprobados
reprobados = 0
PARA i = 0 HASTA GRUPOS - 1 HACER
    PARA o = 0 HASTA ESTUDIANTES - 1 HACER
        SI esc[i][o] < 7 ENTONCES
            reprobados = reprobados + 1
        FIN SI
    FIN PARA
FIN PARA
MOSTRAR reprobados
// Grupo con mayor promedio
grupoMayor = 0
promedioMayor = 0
PARA i = 0 HASTA GRUPOS - 1 HACER
    calcular promedio del grupo i
    SI promedioMayor < promedio del grupo ENTONCES
        promedioMayor = promedio del grupo
        grupoMayor = i
    FIN SI
FIN PARA
MOSTRAR grupoMayor
// Alumnos por encima del promedio general
calcular promedioGeneral
PARA i = 0 HASTA GRUPOS - 1 HACER
    PARA o = 0 HASTA ESTUDIANTES - 1 HACER
```

SI esc[i][o] > promedioGeneral ENTONCES

MOSTRAR grupo

MOSTRAR estudiante

MOSTRAR calificación

FIN SI

FIN PARA

FIN PARA

FIN

Código:

escuela.java:

```
package Unidad_1.Tarea.Ej_3;

import javax.swing.JOptionPane;
public class escuela {
int GRUPOS = 4;
int ESTUDIANTES = 6;
double esc[][];
public escuela() {
this.GRUPOS = 4;
this.ESTUDIANTES = 6;
this.esc = new double[GRUPOS][ESTUDIANTES];
}
public void llenar_arreglo(){
for (int i = 0; i < GRUPOS; i++) {
for (int o = 0; o < ESTUDIANTES; o++) {
double v = 0;
do {
String n = JOptionPane.showInputDialog("Ingresa la calificación del alumno: " + (o + 1) + " del
grupo: " + (i+1));
v = validar(n);
} while (v == -1);
esc[i][o] = v;
}
}
```

```
}
public double validar(String n) {
try {
double x = Double.parseDouble(n);
if (x <= 10.0 && x >= 0.0) {
return x;
} else {
return -1;
}
} catch (Exception e) {
JOptionPane.showMessageDialog(null,"Solo números entre 0 y 10");
return -1;
}
}
public double promedioGrupo(int g){
double prom = 0;
for(int i=0;i<ESTUDIANTES;i++){
prom+=esc[g][i];
}
return (prom/ESTUDIANTES);
}
public double promedioGeneral(){
double prom=0;
for(int i=0;i<GRUPOS;i++){
prom+=promedioGrupo(i);
}
return (prom/GRUPOS);
}
public void califMax(){
double max = 0;
int g=0,e=0;
for(int i=0;i<GRUPOS;i++){
for (int o = 0; o < ESTUDIANTES; o++) {
if(max<esc[i][o]){
max = esc[i][o];
g = i;
e = o;
}
}
}
System.out.println("==== Califiación más alta ===\n" + max);
System.out.println("Grupo: " + (g+1)+"\nEstudiante: " + (e+1));
}
public int aprobaron(){
int aprobados = 0;
```

```
for(int i=0;i<GRUPOS;i++){
for (int o = 0; o < ESTUDIANTES; o++) {
if(esc[i][o]>=7.0){
aprobados++;
}
}
}
return aprobados;
}
public int reprobaron(){
int reprobados = 0;
for(int i=0;i<GRUPOS;i++){
for (int o = 0; o < ESTUDIANTES; o++) {
if(esc[i][o]<7.0){
reprobados++;
}
}
}
return reprobados;
}
public int grupoTop(){
int gr =0;
double prommax=0;
for(int i=0;i<GRUPOS;i++){
double p = promedioGrupo(i);
if(prommax<p){
prommax = p;
gr = i;
}
}
return gr;
}
public String alumnosDestacados(){
String al = "";
double prom = promedioGeneral();
for(int i=0;i<GRUPOS;i++){
for (int o = 0; o < ESTUDIANTES; o++) {
if(esc[i][o]>prom){
al += "Grupo: " + (i + 1) + " | Estudiante: " + (o + 1)+ " | Calificación: " + esc[i][o] + "\n";
}
}
}
return al;
}
}
```

main.java

```
package Unidad_1.Tarea.Ej_3;

public class main {
public static void main(String[] args) {
escuela universidad = new escuela();
universidad.llenar_arreglo();
System.out.println("===== PROMEDIOS POR GRUPO =====");
for (int i = 0; i < 4; i++) {
System.out.println(
"Grupo " + (i + 1) + ": "
+ String.format("%.2f", universidad.promedioGrupo(i))
);
}
System.out.println("===== PROMEDIO GENERAL =====");
System.out.println(
String.format("%.2f", universidad.promedioGeneral())
);
universidad.califMax();
System.out.println("===== APROBADOS Y REPROBADOS =====");
System.out.println("Aprobados: " + universidad.aprobaron());
System.out.println("Reprobados: " + universidad.reprobaron());
System.out.println("===== GRUPO CON MAYOR PROMEDIO =====");
System.out.println("Grupo: " + (universidad.grupoTop() + 1));
System.out.println("===== ESTUDIANTES SOBRE EL PROMEDIO GENERAL =====");
System.out.println(universidad.alumnosDestacados());
}
}
```

Datos de prueba:

- 10
- 10
- 10
- 8.8
- 9.5
- 9.8
- 4.5
- 3.5
- 9.9
- 10
- 10
- 7
- 6.9
- 5.7
- 8.8

- 5.7
- 6.7
- 8.8
- 7.7
- 6.7
- 9.8
- 6.7
- 9.9
- 6.9

Resultado:

```
 Zunny …/Estructura de datos  main ?  20:56
  cd /home/Zunny/Documentos/UTTT/4°\ CUATRI/Estructura\ 
home/Zunny/.config/Code/User/workspaceStorage/0a465d4da33
===== PROMEDIOS POR GRUPO =====
Grupo 1: 9,68 …
Grupo 4: 8,13
===== PROMEDIO GENERAL =====
8,05
==== Califiación más alta ===
10.0
Grupo: 1
Estudiante: 1
===== APROBADOS Y REPROBADOS =====
Aprobados: 15
Reprobados: 9
===== GRUPO CON MAYOR PROMEDIO =====
Grupo: 1
===== ESTUDIANTES SOBRE EL PROMEDIO GENERAL =====
Grupo: 1 | Estudiante: 1 | Calificación: 10.0
Grupo: 1 | Estudiante: 2 | Calificación: 10.0
Grupo: 1 | Estudiante: 3 | Calificación: 10.0
Grupo: 1 | Estudiante: 4 | Calificación: 8.8
Grupo: 1 | Estudiante: 5 | Calificación: 9.5
Grupo: 1 | Estudiante: 6 | Calificación: 9.8
Grupo: 2 | Estudiante: 3 | Calificación: 9.9
Grupo: 2 | Estudiante: 4 | Calificación: 10.0
Grupo: 2 | Estudiante: 5 | Calificación: 10.0
Grupo: 3 | Estudiante: 3 | Calificación: 8.8
Grupo: 4 | Estudiante: 3 | Calificación: 9.9
Grupo: 4 | Estudiante: 4 | Calificación: 8.8
Grupo: 4 | Estudiante: 5 | Calificación: 9.8
```

4. Matriz de ventas

Una empresa registra las ventas de **5 sucursales durante 7 días**.

ventas[5][7]

Las filas representan sucursales y las columnas días.

El programa deberá:

- Capturar las ventas.
- Calcular el total vendido por cada sucursal.
- Calcular el total vendido por cada día.
- Determinar la sucursal con mayores ventas.
- Determinar el día con mayores ventas.
- Calcular el promedio de ventas de toda la empresa.
- Mostrar las sucursales que estén por encima del promedio.
- Identificar el día con menores ventas.

Nota: generar una matriz adicional donde cada fila contenga:

Sucursal | Total | Promedio

Pseudocódigo:

```
INICIO

   DEFINIR SUCURSALES = 5

   DEFINIR DIAS = 7

   CREAR matriz ventas[5][7]

   CREAR matriz reporte[5][3]

   MÉTODO llenar_arreglo()

      PARA i = 0 HASTA SUCURSALES - 1

         PARA o = 0 HASTA DIAS - 1

            REPETIR
```

```
            MOSTRAR "Ingresa las ventas de la sucursal i+1 del día o+1"
            LEER n
            v = validar(n)
        HASTA QUE v sea diferente de -1
        ventas[i][o] = v
    FIN PARA
  FIN PARA
FIN MÉTODO
MÉTODO validar(n)
  INTENTAR
    CONVERTIR n a número decimal y guardar en x
    SI x >= 0 ENTONCES
      RETORNAR x
    SI NO
      RETORNAR -1
    FIN SI
  SI OCURRE UN ERROR ENTONCES
    MOSTRAR "Solo números positivos"
    RETORNAR -1
  FIN INTENTAR
FIN MÉTODO
MÉTODO totalSucursal(s)
```

```
    total = 0
    PARA i = 0 HASTA DIAS - 1
        total = total + ventas[s][i]
    FIN PARA
    RETORNAR total
FIN MÉTODO
MÉTODO totalDia(d)
    total = 0
    PARA i = 0 HASTA SUCURSALES - 1
        total = total + ventas[i][d]
    FIN PARA
    RETORNAR total
FIN MÉTODO
MÉTODO sucursalTop()
    s = 0
    max = 0
    PARA i = 0 HASTA SUCURSALES - 1
        total = totalSucursal(i)
        SI max < total ENTONCES
            max = total
            s = i
        FIN SI
```

```
    FIN PARA
    RETORNAR s
FIN MÉTODO
MÉTODO diaTop()
    d = 0
    max = 0
    PARA i = 0 HASTA DIAS - 1
        total = totalDia(i)
        SI max < total ENTONCES
            max = total
            d = i
        FIN SI
    FIN PARA
    RETORNAR d
FIN MÉTODO
MÉTODO mostrarTotalSucursal()
    s = ""
    PARA i = 0 HASTA SUCURSALES - 1
        s = s + "Sucursal " + (i+1) + ": "
                + totalSucursal(i) + salto de línea
    FIN PARA
    RETORNAR s
```

```
FIN MÉTODO

MÉTODO mostrarTotalDia()
    s = ""
    PARA i = 0 HASTA DIAS - 1
        s = s + "Día " + (i+1) + ": "
                + totalDia(i) + salto de línea
    FIN PARA
    RETORNAR s
FIN MÉTODO

MÉTODO promedio()
    total = 0
    PARA i = 0 HASTA SUCURSALES - 1
        total = total + totalSucursal(i)
    FIN PARA
    promedio = total / (SUCURSALES * DIAS)
    RETORNAR promedio
FIN MÉTODO

MÉTODO sucursalesSobrePromedio()
    s = ""
    prom = promedio()
    PARA i = 0 HASTA SUCURSALES - 1
        total = totalSucursal(i)
```

```
        promSucursal = total / DIAS
        SI promSucursal > prom ENTONCES
            s = s + "Sucursal " + (i+1)
                + " | Promedio: " + promSucursal
                + salto de línea
        FIN SI
    FIN PARA
    RETORNAR s
FIN MÉTODO
MÉTODO diaMenor()
    d = 0
    menor = totalDia(0)
    PARA i = 1 HASTA DIAS - 1
        total = totalDia(i)
        SI total < menor ENTONCES
            menor = total
            d = i
        FIN SI
    FIN PARA
    RETORNAR d
FIN MÉTODO
MÉTODO llenarReporte()
```

```
        PARA i = 0 HASTA SUCURSALES - 1
            totals = totalSucursal(i)
            reporte[i][0] = i
            reporte[i][1] = totals
            reporte[i][2] = totals / DIAS
        FIN PARA
    FIN MÉTODO
    MÉTODO mostrarReporte()
        MOSTRAR "Sucursal | Total | Promedio"
        PARA i = 0 HASTA SUCURSALES - 1
            MOSTRAR reporte[i][0] + 1
                + " | " + reporte[i][1]
                + " | " + reporte[i][2]
        FIN PARA
    FIN MÉTODO
FIN
```

Código:

tiendas.java

```
package Unidad_1.Tarea.Ej_4;

import javax.swing.JOptionPane;
public class tiendas {
```

```
int SUCURSALES = 5;
int DIAS = 7;
double ventas[][];
double reporte[][];

public tiendas() {
this.SUCURSALES = 5;
this.DIAS = 7;
this.ventas = new double[SUCURSALES][DIAS];
this.reporte = new double[SUCURSALES][3];
}

public void llenar_arreglo(){
for(int i=0;i<SUCURSALES;i++){
for(int o=0;o<DIAS;o++){
double v=0;
do{
String n = JOptionPane.showInputDialog("Ingresa las ventas de la sucursal: "+ (i+1) + " del día: "
+ (o+1));
v = validar(n);
}while(v==-1);
ventas[i][o] = v;
}
}
}

public double validar(String n){
try{
double x = Double.parseDouble(n);
if(x>=0){
return x;
}else{
return -1;
}
}catch(Exception e){
JOptionPane.showMessageDialog(null,"Solo números positivos");
return -1;
}
}

public double totalSucursal(int s){
double total=0;
for(int i=0;i<DIAS;i++){
total+=ventas[s][i];
```

```
}
return total;
}
public double totalDia(int d){
double total=0;
for(int i=0;i<SUCURSALES;i++){
total+=ventas[i][d];
}
return total;
}

public int sucursalTop(){
int s=0;
double max=0;
for(int i=0;i<SUCURSALES;i++){
double total = totalSucursal(i);
if(max<total){
max=total;
s=i;
}
}
return s;
}

public int diaTop(){
int d=0;
double max=0;
for(int i=0;i<DIAS;i++){
double total = totalDia(i);
if(max<total){
max=total;
d=i;
}
}
return d;
}

public String mostrarTotalSucursal(){
String s="";
for(int i=0;i<SUCURSALES;i++){
s+="Sucursal " + (i+1) + ": "
+ String.format("%.2f",totalSucursal(i)) + "\n";
}
return s;
```

```
}
public String mostrarTotalDia(){
String s="";
for(int i=0;i<DIAS;i++){
s+="Día " + (i+1) + ": "
+ String.format("%.2f",totalDia(i)) + "\n";
}
return s;
}
public double promedio(){
double total=0;
for(int i=0;i<SUCURSALES;i++){
total+=totalSucursal(i);
}
return total/(SUCURSALES*DIAS);
}
public String sucursalesSobrePromedio(){
String s="";
double prom=promedio();
for(int i=0;i<SUCURSALES;i++){
double total=totalSucursal(i);
double promSucursal=total/DIAS;
if(promSucursal>prom){
s+="Sucursal: " + (i+1)+ " | Promedio: " + String.format("%.2f",promSucursal)+ "\n";
}
}
return s;
}
public int diaMenor(){
int d=0;
double menor=totalDia(0);
for(int i=1;i<DIAS;i++){
double total=totalDia(i);
if(total<menor){
menor=total;
d=i;
}
}
return d;
}
public void llenarReporte(){
for(int i = 0; i < SUCURSALES; i++){
double totals = totalSucursal(i);
reporte[i][0] = i;
reporte[i][1] = totals;
```

```
reporte[i][2] = totals/DIAS;

}
}
public void mostrarReporte(){
System.out.println("Sucursal | Total | Promedio");
for(int i=0;i<SUCURSALES;i++){
System.out.println(
(int)(reporte[i][0]+1) + " | "+ String.format("%.2f",reporte[i][1])+ " | "+
String.format("%.2f",reporte[i][2]));
}
}
}
```

main.java

```
package Unidad_1.Tarea.Ej_4;

public class main {
public static void main(String[] args) {
tiendas negocio = new tiendas();
negocio.llenar_arreglo();
negocio.llenarReporte();
System.out.println("===== TOTAL POR SUCURSAL =====");
System.out.println(negocio.mostrarTotalSucursal());
System.out.println("===== TOTAL POR DÍA =====");
System.out.println(negocio.mostrarTotalDia());
System.out.println("===== SUCURSAL CON MAYORES VENTAS =====");
System.out.println("Sucursal: " + (negocio.sucursalTop()+1));
System.out.println("===== DÍA CON MAYORES VENTAS =====");
System.out.println("Día: " + (negocio.diaTop()+1));
System.out.println("===== PROMEDIO DE VENTAS =====");
System.out.println(String.format("%.2f",negocio.promedio()));
System.out.println("===== SUCURSALES SOBRE EL PROMEDIO =====");
System.out.println(negocio.sucursalesSobrePromedio());
System.out.println("===== DÍA CON MENORES VENTAS =====");
System.out.println("Día: " + (negocio.diaMenor()+1));
System.out.println("===== REPORTE =====");
negocio.mostrarReporte();
}
}
```

Datos de prueba

- 1500
- 3000
- 10000
- 21000
- 5620
- 9999
- 10900
- 11500
- 12500
- 20000
- 20001
- 16000
- 12000
- 18000
- 9200
- 16500
- 11540
- 12222
- 5500
- 16400
- 9000
- 5600
- 7000
- 9800
- 16700
- 21000
- 11000
- 10666
- 7676
- 6767
- 6996
- 6969
- 10000
- 7000
- 9500

Resultado:

```
 Zunny …/Estructura de datos  main ?  21:07
 cd /home/Zunny/Documentos/UTTT/4°\ CUATRI/Estructura\
home/Zunny/.config/Code/User/workspaceStorage/0a465d4da33
===== TOTAL POR SUCURSAL =====
Sucursal 1: 62019,00
Sucursal 2: 110001,00
Sucursal 3: 80362,00
Sucursal 4: 81766,00
Sucursal 5: 54908,00

===== TOTAL POR DÍA =====
Día 1: 35476,00
Día 2: 45767,00
Día 3: 58336,00
Día 4: 76892,00
Día 5: 58120,00
Día 6: 56399,00
Día 7: 58066,00

===== SUCURSAL CON MAYORES VENTAS =====
Sucursal: 2
===== DÍA CON MAYORES VENTAS =====
Día: 4
===== PROMEDIO DE VENTAS =====
11115,89
===== SUCURSALES SOBRE EL PROMEDIO =====
Sucursal: 2 | Promedio: 15714,43
Sucursal: 3 | Promedio: 11480,29
Sucursal: 4 | Promedio: 11680,86
===== DÍA CON MENORES VENTAS =====
Día: 1
===== REPORTE =====
Sucursal | Total | Promedio
1 | 62019,00 | 8859,86
2 | 110001,00 | 15714,43
3 | 80362,00 | 11480,29
4 | 81766,00 | 11680,86
5 | 54908,00 | 7844,00
```

5. Matriz de asientos de un cine

Representa una sala de cine mediante una matriz de:

8 filas × 10 columnas

Cada posición representa un asiento:

0 = disponible

1 = ocupado

El programa deberá permitir:

- Mostrar el mapa de asientos.
- Reservar un asiento.
- Cancelar una reserva.
- Consultar si un asiento está disponible.
- Contar los asientos disponibles.
- Contar los asientos ocupados.
- Mostrar el porcentaje de ocupación.
- Determinar cuál fila tiene mayor ocupación.
- Mostrar todas las filas completamente ocupadas.

Nota: impedir que el usuario reserve un asiento ya ocupado.

Pseudocódigo:

INICIO

DEFINIR F = 8

DEFINIR C = 10

CREAR matriz sala[8][10]

INICIALIZAR todos los elementos de sala en 0

MÉTODO mostrarAsientos()

```
    PARA i = 0 HASTA F - 1
        PARA o = 0 HASTA C - 1
            MOSTRAR sala[i][o]
        FIN PARA
        MOSTRAR salto de línea
    FIN PARA
FIN MÉTODO
MÉTODO reservarAsiento(fi, co)
    SI sala[fi][co] == 0 ENTONCES
        sala[fi][co] = 1
        MOSTRAR "El asiento ha sido reservado con éxito"
    SI NO
        MOSTRAR "El asiento ya está reservado"
    FIN SI
FIN MÉTODO
MÉTODO cancelarReserva(fi, co)
    SI sala[fi][co] == 1 ENTONCES
        sala[fi][co] = 0
        MOSTRAR "El asiento ya no está reservado"
    SI NO
        MOSTRAR "El asiento no estaba reservado"
    FIN SI
```

```
FIN MÉTODO

MÉTODO consultarReserva(fi, co)
    SI sala[fi][co] == 1 ENTONCES
        MOSTRAR "El asiento ya está reservado"
    SI NO
        MOSTRAR "El asiento no está reservado"
    FIN SI
FIN MÉTODO

MÉTODO asientosDisp()
    disp = 0
    PARA i = 0 HASTA F - 1
        PARA o = 0 HASTA C - 1
            SI sala[i][o] == 0 ENTONCES
                disp = disp + 1
            FIN SI
        FIN PARA
    FIN PARA
    RETORNAR disp
FIN MÉTODO

MÉTODO asientosOcup()
    ocup = 0
    PARA i = 0 HASTA F - 1
```

```
        PARA o = 0 HASTA C - 1
            SI sala[i][o] == 1 ENTONCES
                ocup = ocup + 1
            FIN SI
        FIN PARA
    FIN PARA
    RETORNAR ocup
FIN MÉTODO
MÉTODO porcentajeOcupacion()
    porcentaje = (asientosOcup() * 100) / 80
    RETORNAR porcentaje
FIN MÉTODO
MÉTODO filaMasOcup()
    filamax = 0
    sumMax = 0
    PARA i = 0 HASTA F - 1
        sum = 0
        PARA o = 0 HASTA C - 1
            sum = sum + sala[i][o]
        FIN PARA
        SI sumMax < sum ENTONCES
            sumMax = sum
```

```
            filamax = i
        FIN SI
    FIN PARA
    RETORNAR filamax
  FIN MÉTODO
  MÉTODO mostrarFilasOcup()
    mensaje = ""
    PARA i = 0 HASTA F - 1
      sum = 0
      PARA o = 0 HASTA C - 1
        sum = sum + sala[i][o]
      FIN PARA
      SI sum == 10 ENTONCES
        mensaje = mensaje + "Fila: " + (i+1) + " Llena"
      FIN SI
    FIN PARA
    RETORNAR mensaje
  FIN MÉTODO
FIN
```

Código:

cinepolis.java

```java
package Unidad_1.Tarea.Ej_5;


import javax.swing.JOptionPane;

public class cinepolis {
int F = 8;
int C = 10;
int sala [][];
public cinepolis(){
this.F = 8;
this.C =10;
this.sala = new int[F][C];
}

public void mostrarAsientos(){
for(int i = 0; i<F;i++){
for(int o = 0; o<C;o++){
System.out.print(sala[i][o] + " ");
}
System.out.print("\n");
}
}
public void reservarAsiento(int fi, int co){
if(sala[fi][co] == 0){
sala[fi][co] = 1;
System.out.println("El asiento: ["+fi+"]["+co+"] ha sido reservado con éxito");
}else{
System.out.println("Lo lamentamos, el asiento: ["+fi+"]["+co+"] Ya está reservado");
}
}
public void cancelarReserva(int fi, int co){
if(sala[fi][co] == 1){
sala[fi][co] = 0;
System.out.println("El asiento: ["+fi+"]["+co+"] ya no está reservado");
}else{
System.out.println("El asiento: ["+fi+"]["+co+"] no estaba reservado");
}
}

public void consultarReserva(int fi, int co){
if(sala[fi][co] == 1){
System.out.println("El asiento: ["+fi+"]["+co+"] ya esta reservado");
}else{
System.out.println("El asiento: ["+fi+"]["+co+"] no esta reservado");
```

```
}
}
public int asientosDisp(){
int disp=0;
for(int i = 0; i < F; i++){
for(int o = 0; o< C; o++){
if(sala[i][o] == 0){
disp++;
}
}
}
return disp;
}

public int asientosOcup(){
int ocup=0;
for(int i = 0; i < F; i++){
for(int o = 0; o< C; o++){
if(sala[i][o] == 1){
ocup++;
}
}
}
return ocup;
}

public double porcentajeOcupacion(){
return (asientosOcup()*100.0)/80;
}
public int filaMasOcup(){
int filamax =0;
int sumMax = 0;
int sum = 0;
for(int i = 0; i < F;i++){
sum=0;
for(int o = 0; o<C;o++){
sum+= sala[i][o];
}
if(sumMax<sum){
sumMax=sum;
filamax = i;
}
}
return filamax;
}
```

```
public String mostrarFilasOcup(){
int sum;
String mensaje="";
for(int i = 0; i < F;i++){
sum=0;
for(int o = 0; o<C;o++){
sum+= sala[i][o];
}
if(sum == 10){
mensaje += "Fila: " + (i+1) + " Llena";
}
}
return mensaje;
}
}
```

main.java

```
package Unidad_1.Tarea.Ej_5;


import javax.swing.JOptionPane;

public class main {

public static void main(String[] args) {

String opcion;

String menu = "MENU CINE\n"
+ "1. Mostrar mapa de asientos\n"
+ "2. Reservar asiento\n"
+ "3. Cancelar reserva\n"
+ "4. Consultar asiento\n"
+ "5. Contar asientos disponibles\n"
+ "6. Contar asientos ocupados\n"
+ "7. Mostrar porcentaje de ocupación\n"
+ "8. Mostrar fila con mayor ocupación\n"
+ "9. Mostrar filas completamente ocupadas\n"
+ "10. Salir";
```

```
cinepolis cine = new cinepolis();
while(true){
opcion = JOptionPane.showInputDialog(menu);
switch(opcion){
case "1":
System.out.println("===== MAPA DE ASIENTOS =====");
cine.mostrarAsientos();
break;
case "2":
int fi = Integer.parseInt(
JOptionPane.showInputDialog("Ingresa la fila (1-8)")
);
int co = Integer.parseInt(
JOptionPane.showInputDialog("Ingresa el asiento (1-10)")
);
cine.reservarAsiento(fi-1,co-1);
break;
case "3":
fi = Integer.parseInt(
JOptionPane.showInputDialog("Ingresa la fila (1-8)")
);
co = Integer.parseInt(
JOptionPane.showInputDialog("Ingresa el asiento (1-10)")
);
cine.cancelarReserva(fi-1,co-1);
break;
case "4":
fi = Integer.parseInt(
JOptionPane.showInputDialog("Ingresa la fila (1-8)")
);
co = Integer.parseInt(
JOptionPane.showInputDialog("Ingresa el asiento (1-10)")
);
cine.consultarReserva(fi-1,co-1);
break;
case "5":
JOptionPane.showMessageDialog(
null,
"Asientos disponibles: " + cine.asientosDisp()
);
break;
case "6":
JOptionPane.showMessageDialog(
null,
"Asientos ocupados: " + cine.asientosOcup()
```

```
);
break;
case "7":
JOptionPane.showMessageDialog(
null,
"Porcentaje de ocupación: "
+ String.format("%.2f",cine.porcentajeOcupacion()) + "%"
);
break;
case "8":
JOptionPane.showMessageDialog(
null,
"Fila con mayor ocupación: "
+ (cine.filaMasOcup()+1)
);
break;
case "9":
JOptionPane.showMessageDialog(
null,
"Filas completamente ocupadas:\n"
+ cine.mostrarFilasOcup()
);
break;
case "10":
System.exit(0);
break;
default:
JOptionPane.showMessageDialog(
null,
"Opción no válida"
);
break;
}
}
}
```

resultado:

```
 Zunny …/Estructura de datos  main ?  21:39
 cd /home/Zunny/Documentos/UTTT/4°\ CUATRI/Estructura\
home/Zunny/.config/Code/User/workspaceStorage/0a465d4da33
===== MAPA DE ASIENTOS =====
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
El asiento: [1][5] ha sido reservado con éxito
El asiento: [1][6] ha sido reservado con éxito
El asiento: [1][8] ha sido reservado con éxito
El asiento: [1][8] ya no está reservado
===== MAPA DE ASIENTOS =====
0 0 0 0 1 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
El asiento: [1][5] ya esta reservado
===== MAPA DE ASIENTOS =====
0 0 0 0 1 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
```



6. Análisis de una matriz numérica

Genera una matriz de **6 × 6** con números enteros.

El programa deberá determinar:

- La suma de todos los elementos.
- La suma de cada fila.
- La suma de cada columna.
- El número mayor y su posición.
- El número menor y su posición.
- La suma de la diagonal principal.
- La suma de la diagonal secundaria.
- Cuántos números pares existen.
- Cuántos números impares existen.
- Si la matriz es **simétrica**.

Una matriz es simétrica cuando:

matriz[i][j] == matriz[j][i]

Nota:desarrollar un método llamado:

esSimetrica()

que devuelva true o false.

Pseudocódigo

```
INICIO
    DEFINIR FILAS = 6
    DEFINIR COLUMNAS = 6
    CREAR matriz mat[6][6]
    MÉTODO llenar_arreglo()
        PARA i = 0 HASTA FILAS - 1
            PARA o = 0 HASTA COLUMNAS - 1
                MOSTRAR "Ingresa el número de la posición [i][o]"
                LEER n
                CONVERTIR n A ENTERO
                mat[i][o] = n
            FIN PARA
        FIN PARA
    FIN MÉTODO
    MÉTODO mostrarMatriz()
        PARA i = 0 HASTA FILAS - 1
            PARA o = 0 HASTA COLUMNAS - 1
                MOSTRAR mat[i][o]
            FIN PARA
```

```
        MOSTRAR salto de línea
    FIN PARA
FIN MÉTODO
MÉTODO sumaTotal()
    suma = 0
    PARA i = 0 HASTA FILAS - 1
        PARA o = 0 HASTA COLUMNAS - 1
            suma = suma + mat[i][o]
        FIN PARA
    FIN PARA
    RETORNAR suma
FIN MÉTODO
MÉTODO sumaFila(f)
    suma = 0
    PARA i = 0 HASTA COLUMNAS - 1
        suma = suma + mat[f][i]
    FIN PARA
    RETORNAR suma
FIN MÉTODO
MÉTODO sumaColumna(c)
    suma = 0
    PARA i = 0 HASTA FILAS - 1
```

```
            suma = suma + mat[i][c]
        FIN PARA
        RETORNAR suma
    FIN MÉTODO
    MÉTODO mayor()
        m = mat[0][0]
        PARA i = 0 HASTA FILAS - 1
            PARA o = 0 HASTA COLUMNAS - 1
                SI m < mat[i][o] ENTONCES
                    m = mat[i][o]
                FIN SI
            FIN PARA
        FIN PARA
        RETORNAR m
    FIN MÉTODO
    MÉTODO posicionMayor()
        pos = ""
        m = mayor()
        PARA i = 0 HASTA FILAS - 1
            PARA o = 0 HASTA COLUMNAS - 1
                SI mat[i][o] == m ENTONCES
                    pos = pos + "[i][o]" + salto de línea
```

```
            FIN SI
        FIN PARA
    FIN PARA
    RETORNAR pos
FIN MÉTODO
MÉTODO menor()
    m = mat[0][0]
    PARA i = 0 HASTA FILAS - 1
        PARA o = 0 HASTA COLUMNAS - 1
            SI m > mat[i][o] ENTONCES
                m = mat[i][o]
            FIN SI
        FIN PARA
    FIN PARA
    RETORNAR m
FIN MÉTODO
MÉTODO posicionMenor()
    pos = ""
    m = menor()
    PARA i = 0 HASTA FILAS - 1
        PARA o = 0 HASTA COLUMNAS - 1
            SI mat[i][o] == m ENTONCES
```

```
                pos = pos + "[i][o]" + salto de línea
            FIN SI
        FIN PARA
    FIN PARA
    RETORNAR pos
FIN MÉTODO
MÉTODO diagonalPrincipal()
    suma = 0
    PARA i = 0 HASTA FILAS - 1
        suma = suma + mat[i][i]
    FIN PARA
    RETORNAR suma
FIN MÉTODO
MÉTODO diagonalSecundaria()
    suma = 0
    PARA i = FILAS - 1 HASTA 0 CON PASO -1
        suma = suma + mat[i][FILAS - i - 1]
    FIN PARA
    RETORNAR suma
FIN MÉTODO
MÉTODO pares()
    p = 0
```

```
    PARA i = 0 HASTA FILAS - 1
        PARA o = 0 HASTA COLUMNAS - 1
            SI mat[i][o] MOD 2 == 0 ENTONCES
                p = p + 1
            FIN SI
        FIN PARA
    FIN PARA
    RETORNAR p
FIN MÉTODO
MÉTODO impares()
    p = 0
    PARA i = 0 HASTA FILAS - 1
        PARA o = 0 HASTA COLUMNAS - 1
            SI mat[i][o] MOD 2 != 0 ENTONCES
                p = p + 1
            FIN SI
        FIN PARA
    FIN PARA
    RETORNAR p
FIN MÉTODO
MÉTODO esSimetrica()
    PARA i = 0 HASTA FILAS - 1
```

PARA o = 0 HASTA COLUMNAS - 1

SI mat[i][o] != mat[o][i] ENTONCES

RETORNAR FALSO

FIN SI

FIN PARA

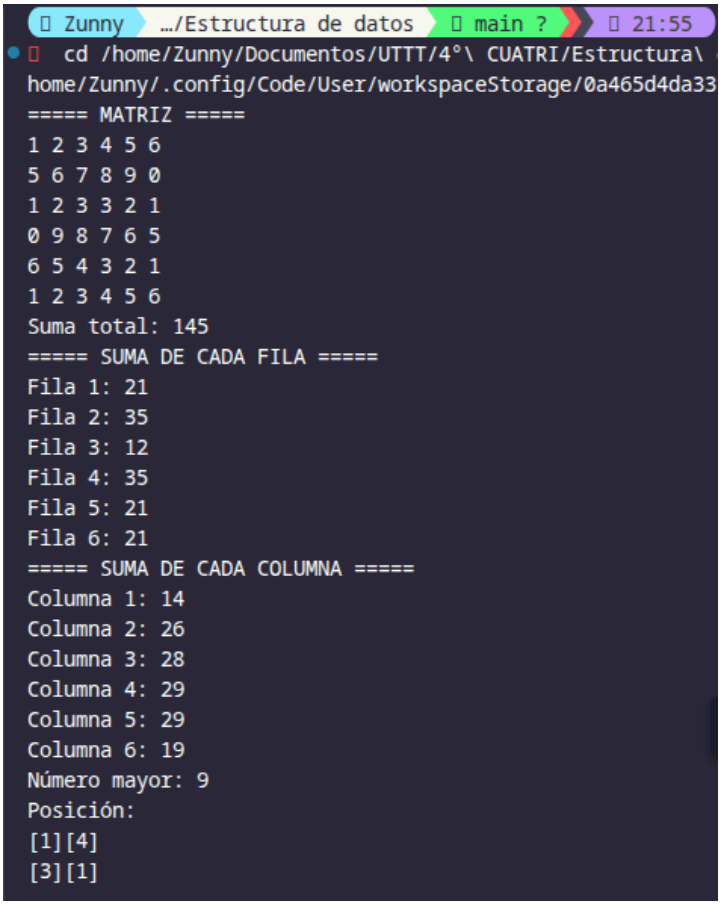
FIN PARA

RETORNAR VERDADERO

FIN MÉTODO

FIN

Resultado:



```
Número menor: 0
Posición:
[1][5]
[3][0]

Suma de la diagonal principal: 25
Suma de la diagonal secundaria: 32
Números pares: 17
Números impares: 19
La matriz no es simétrica
```